

* Always Remember $\rightarrow$ Try Your Best to come up with the Recursive logic only, Because after that DP is just 5 minutes of work. - vHixem

* Problem: -

You are climbing a staircase. It takes n steps to reach the top.

Each time you can either climb 1 or 2 steps. In how many distinct ways can you climb to the top?

Input: $n = 3$

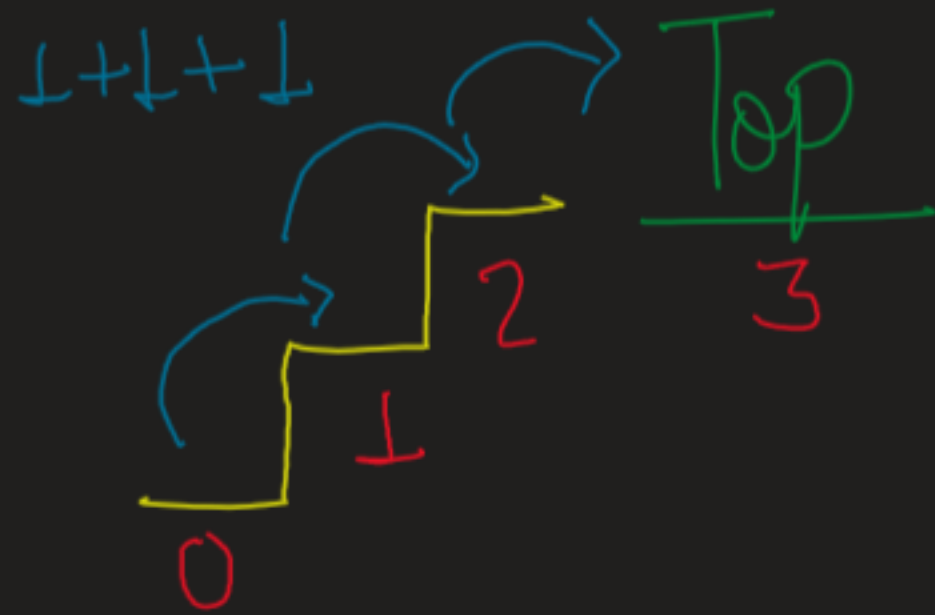
Output: 3

Explanation: There are three ways to climb to the top.

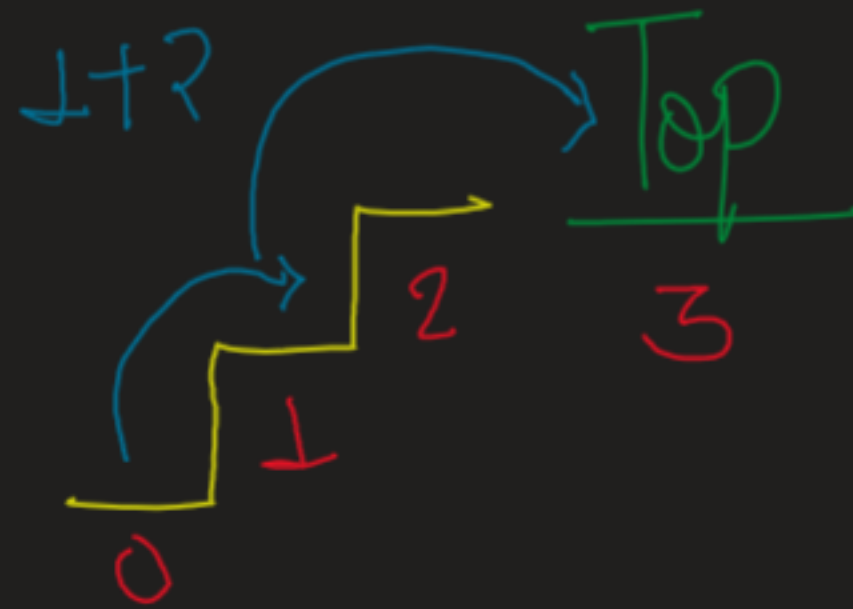
1. 1 step + 1 step + 1 step

2. 1 step + 2 steps

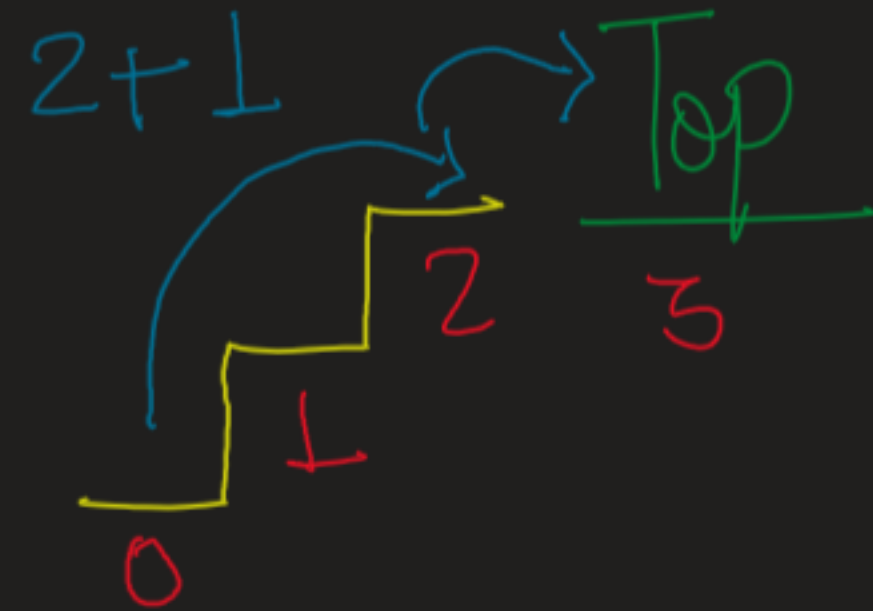
3. 2 steps + 1 step



1 way



1 way



1 way = 3 ways

* Time $\rightarrow O(2^N)$

, $n=4$ | 4, 3, 2, 1

$\rightarrow$ For any value we're exploring two options / possibilities.

Why It's taking Exponential time?



$\rightarrow$ Overlapping Sub problems (Repetitive work)

$\rightarrow$ Means when n is increased, the amount of sub-problem increases

$\rightarrow$ Solving this repetitive problems again and again is increasing time.

$\rightarrow$ This concludes that we could optime time in our recursion.

$\rightarrow$ Now this hints to DP.

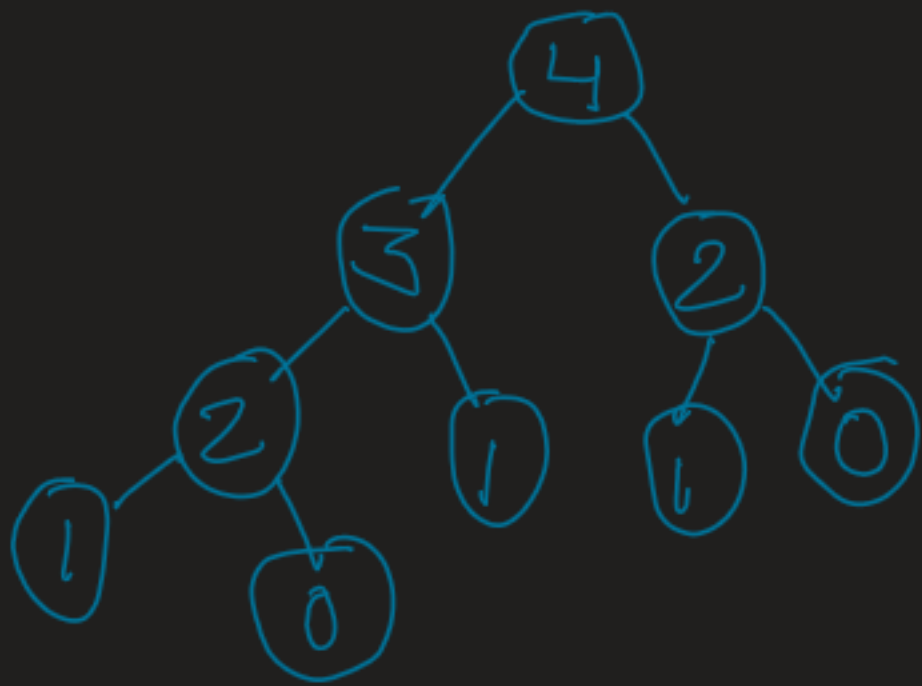
$\rightarrow$ So for recursion, we apply Top-Down DP,

So lets do Memorization.

* let's identify Base cases:-

→ if ($n == 1$) return 1

→ if ($n == 0$) return 1



* For test part:-

(we just need previous two counts):

→ int count1 (find recursively)

int count2 (find recursively)

→ return sum of counts

~ Recursive code:-

```
int ways (int n){  
    if (n==0) return 1;  
    if (n==1) return 1;  
    int JumpstepDown = ways(n-1);  
    int Jump2stepDown = ways(n-2);  
    return JumpstepDown + Jump2stepDown;  
}
```

→ Now memoize it (Apply DP)
→ let's code it

